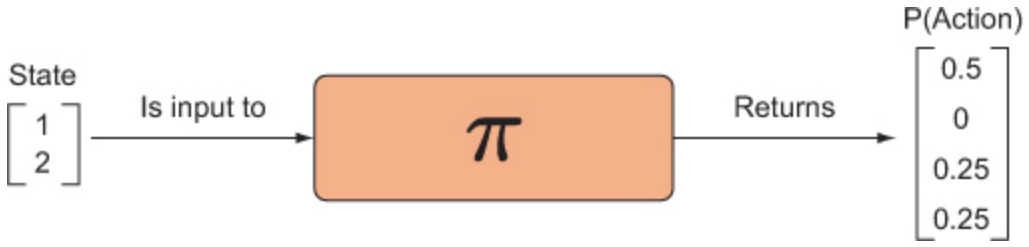
choose action 2, but sometimes we'll grab another action, and that gives us the opportunity to explore. Using this analogy, every time the state of the environment changes, we give the state to our policy network, and it uses that to fill the jar with a new set of labeled notes representing the actions in different proportions. Then we randomly pick from the jar.

This class of algorithms is called *policy gradient methods*, and it has a few important differences from the DQN algorithm; we'll explore these differences in this chapter. Policy gradient methods offer a few advantages over value prediction methods like DQN. One is that, as we already discussed, we no longer have to worry about devising an action-selection strategy like epsilon-greedy; instead, we directly sample actions from the policy. Remember, we spent a lot of time cooking up methods to improve the stability of training our DQN—we had to use experience replay and target networks, and there are a number of other methods in the academic literature that we could have used. A policy network tends to simplify some of that complexity.

4.1.2. Stochastic policy gradient

There are many different flavors of policy gradient methods. We will start with the *stochastic policy gradient* method (figure 4.3), which is what we just described. With a stochastic policy gradient, the output of our neural network is an action vector that represents a probability distribution.

Figure 4.3. A stochastic policy function. A policy function accepts a state and returns a probability distribution over actions. It is stochastic because it returns a probability distribution over actions rather than returning a deterministic, single action.



The policy we'll follow is selecting an action from this probability distribution. This means that if our agent ends up in the same state twice,